

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования

Кафедра индустриального программирования

ОТЧЁТ

по практическому занятию № 1

по дисциплине «Создание программного обеспечения»

Тема: «Введение в дисциплину. Организация командной работы и выбор
проекта»

Выполнил:

студент группы ЭФБО-17-24

Васин С. А. _____

Проверил:

Макиевский С. Е. _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Цель работы	3
Задание	3
1 ХОД РАБОТЫ	4
1.1 Структура семестра и оценивание	4
1.2 Роли и организация работы	5
1.3 Выбор темы и исследование	5
1.4 Видение, возможности и границы	6
1.5 Репозиторий и процесс разработки	8
ЗАКЛЮЧЕНИЕ	9
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	9

ВВЕДЕНИЕ

Цель работы

Изучить структуру семестра и систему оценивания, определить роли и процесс работы, выбрать тему проекта по результатам исследования аналогов, сформулировать видение продукта и организовать репозиторий.

Задание

1. Рассмотреть командную, исследовательскую и проектную составляющие семестра, недельные блоки и балльную систему.
2. Описать пять ролей, их ответственность и результаты работы.
3. Выбрать тему, сравнить аналоги и сформулировать исследовательские вопросы.
4. Определить возможности, границы и предварительный стек продукта.
5. Подготовить структуру репозитория, правила разработки и критерии готовности.

1 ХОД РАБОТЫ

1.1 Структура семестра и оценивание

Семестр объединяет **командную работу, исследование и разработку**: распределение ответственности, проверку проектных гипотез и создание продукта. Для GPU Rent это организация пяти ролей, исследование корректного выделения GPU и биллинга, реализация БД и дальнейшая разработка веб-приложения.

План сгруппирован по блокам исходного задания.

Таблица 1 — План по неделям

Недели	Темы практик и результаты GPU Rent
1–2	Введение, требования и планирование: тема, роли, видение, сценарии, стек
3–4, 9–10	Проектирование БД и требований к данным: модель, миграции, ограничения, объёмы и сроки хранения
5–6	Бэкенд и фронтенд: серверная логика и первые экраны
7–8, 13–14	Деплой и окружение: воспроизводимая выкладка, HTTPS, резервное копирование
11–12	RESTful API: спецификация, ошибки, пагинация, идиоматичность
15–20	Архитектура, интерфейс, БД, сервер, API, аутентификация: курсовой проект и бизнес-правила
21–24	Интеграция, безопасность, кэширование, паттерны: связка компонентов и оптимизация
25–30	Тестирование, развёртывание, CI/CD, мониторинг, обратная связь, масштабирование: проверки и эксплуатация
31–32	Итоги: демонстрация и защита

Задание предусматривает **пять практических задач**. П1 и П2 оцениваются по пятибалльной системе. Для П3 и П4 применяется комплексная оценка качества работы над курсовым проектом и ответов преподавателю. Защита включает дополнительные вопросы, поэтому решения должны быть объяснимыми, а результаты — воспроизводимыми по исходникам и инструкциям.

1.2 Роли и организация работы

Проект выполняет один студент — Васин С. А., группа ЭФБО-17-24. Он совмещает все пять ролей; отдельные участники команды не назначены. Задание требует командной работы, поэтому индивидуальный формат остаётся организационным ограничением: факт согласования этого формата с преподавателем здесь не утверждается.

Таблица 2 — Ответственность и артефакты ролей

Роль	Ответственность	Артефакты
Тимлид	Приоритеты, сроки, границы, приёмка этапов	План, решения, правила процесса
Системный аналитик	Требования, сценарии, модель данных	Перечень требований, диаграммы, словарь данных
Тестировщик	Критерии приёмки, проверка БД и API	Тест-план, SQL-тесты, результаты замеров
Backend-разработчик	Миграции, бизнес-правила, API, безопасность	SQL-объекты, тестовые данные, серверный код
Frontend-разработчик	Экраны клиента и администратора, интеграция API	Прототипы, компоненты React, стили

Работа ведётся недельными итерациями: планирование, выполнение, проверка и разбор проблем. Исполнитель последовательно действует как аналитик, разработчик и тестировщик; тимлид проверяет полноту этапа. Одновременно ведётся не более двух задач. Решения фиксируются в отчётах. Самопроверка после паузы не заменяет независимого ревью. До внедрения CI проверки БД выполняются вручную; демонстрация преподавателю дополняет контроль.

1.3 Выбор темы и исследование

Тема — **GPU Rent, монолитное веб-приложение аренды облачных GPU-серверов**. Исследователям, студентам, разработчикам и специалистам по графике GPU нужны для обучения моделей, инференса, рендеринга и экспериментов. При кратковременной или нерегулярной нагрузке аренда позволяет использовать оборудование без его покупки и обслуживания.

Сравнение основано на публичных страницах и документации, использованных в исходном исследовании на 30.09.2026 [1–3]. Практическое тестирование сервисов не проводилось. Цены не сопоставляются: они зависят от оборудования, региона и тарифа.

Таблица 3 — Три аналога

Сервис	Существенные особенности	Решение для проекта
RunPod [1]	GPU-поды, шаблоны окружений, диски и сетевые тома, API	Запуск из шаблона и постоянные тома
Vast.ai [2]	Площадка сторонних хостов; посекундная тарификация; хранение оплачивается и после остановки	Точный учёт времени и отдельный учёт хранения
Immers.cloud [3]	Виртуальные машины с GPU; посекундная оплата по заявлению сайта; готовые образы, API OpenStack	Каталог GPU и готовых окружений

Выбор темы позволяет изучить транзакции, конкурентный доступ, временные диапазоны и безопасность данных. Сформулированы шесть исследовательских вопросов:

1. Как исключить одновременное выделение одной GPU?
2. Как обеспечить посекундный биллинг без повторных списаний?
3. Как масштабировать хранение журнала потребления?
4. Как согласовать баланс с журналом денежных операций?
5. Как исключить пересечение периодов цен?
6. Как изолировать данные разных клиентов?

Гипотезы проверяются в ПЗ–П4 миграциями, конкурентными тестами и замерами запросов: используются ограничения и диапазоны, транзакции, секционирование и Row Level Security PostgreSQL [4].

1.4 Видение, возможности и границы

Пользователь пополняет баланс, выбирает GPU и дата-центр, запускает инстанс из шаблона PyTorch, Jupyter или Stable Diffusion, подключается по SSH/Jupyter и оплачивает фактическое время работы. Администратор управляет оборудованием и ценами, отслеживает загрузку и выручку.

Предварительный MVP включает регистрацию и вход, SSH-ключи, каталог GPU и шаблонов, создание, остановку и терминацию инстанса, посекундный биллинг, пополнение и историю баланса, автоостановку при исчерпании средств, управление дата-центрами, нодами, GPU и ценами. API-ключи, сетевые тома, блокировка пользователей, дашборд и аудит отнесены к последующим версиям. Приоритеты уточняются в П2.

Реальная виртуализация не реализуется: команды запуска и остановки имитирует агент ноды. Интеграция с ЮKassa предполагает обработку webhook без реальных платежей. Предусмотрены тарифы on-demand и spot;

правила вытеснения spot-инстансов уточняются в П2. Торговая площадка хостов, Windows-окружения, S3-хранилище и мобильное приложение находятся за границами проекта; веб-интерфейс планируется адаптивным.

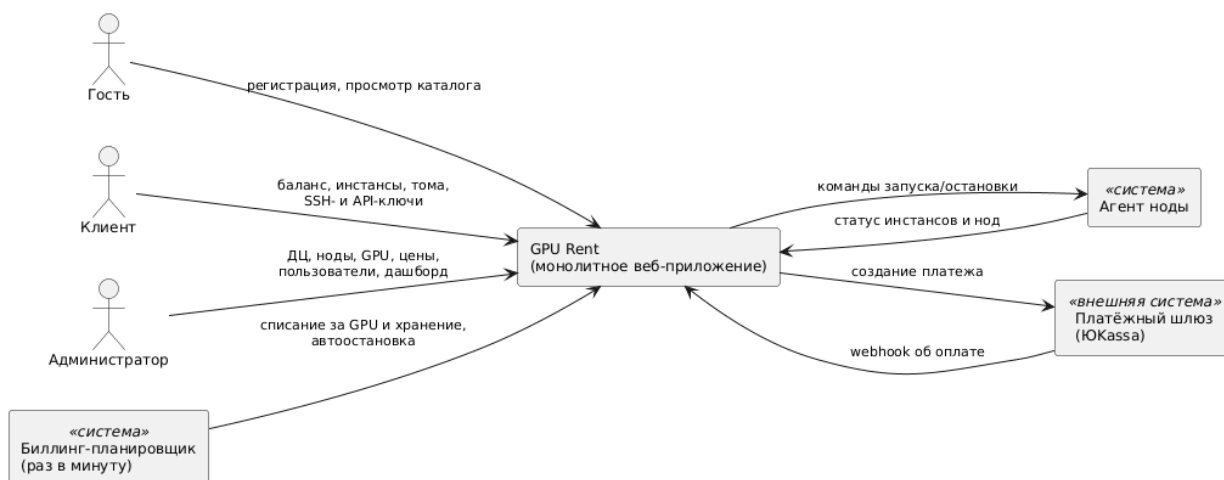


Рисунок 1 — Контекст системы GPU Rent

Контекст включает гостя, клиента, администратора, биллинг-планировщик, агент ноды и внешний платёжный шлюз. Планировщик запускается раз в минуту, при этом стоимость рассчитывается по секундам фактического интервала.

Предварительный стек: монолит на Node.js 24 LTS, TypeScript и Express 4; React 18 с Vite; PostgreSQL 16; Redis 7 для кэша и сессий. Схема хранится в SQL-миграциях, доступ к данным планируется через node-postgres или Kysely. Монолит упрощает развёртывание и транзакции, PostgreSQL предоставляет нужные ограничения и секционирование. Деплой планируется на VPS через Docker Compose и Nginx с TLS; Nginx раздаёт клиентскую статику, Express обслуживает /api.

1.5 Репозиторий и процесс разработки

В `docs/reports/` хранятся отчёты, `docs/diagrams/` — диаграммы, `db/` — миграции, данные, тесты и замеры, `tools/` — сборка, `reports/` — PDF; README содержит инструкции. БД реализована, сервер и клиент запланированы. Сейчас работа ведётся в `main` без CI; далее планируются ветки задач и `pull request`. Коммиты оформляются по Conventional Commits [5]. Готовность означает выполнение требования, применение миграций на чистой БД, успешные тесты, актуальную документацию, отсутствие секретов и воспроизводимость по README. Изменения проверяются перед слиянием; исходники SQL, Markdown и PlantUML сохраняются.

ЗАКЛЮЧЕНИЕ

Изучены структура семестра и оценивание, определены роли и процесс. Выбрана тема GPU Rent, сравнены три аналога, сформулированы исследовательские вопросы, видение, границы и стек. Репозиторий организован. Далее требования уточняются в П2. Ограничения: индивидуальный формат, отсутствие независимого ревью и практической проверки аналогов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Runpod Documentation: Pods overview. — URL: <https://docs.runpod.io/pods/overview> (дата обращения: 30.09.2026).
2. Vast.ai Documentation: Pricing. — URL: <https://docs.vast.ai/documentation/instances/pricing> (дата обращения: 30.09.2026).
3. Immers.cloud: облачные серверы с GPU. — URL: <https://immers.cloud> (дата обращения: 30.09.2026).
4. PostgreSQL 16 Documentation. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 30.09.2026).
5. Conventional Commits 1.0.0. — URL: <https://www.conventionalcommits.org/ru/v1.0.0/> (дата обращения: 30.09.2026).